Expert Systems With Applications 291 (2025) 128411

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Meta-relation-based heterogeneous graph neural network with deep reinforcement learning for flexible job shop scheduling

Yuzhi Zhang, Shidu Dong *, Zhenfang Yuan, Ting Wen, Jianfeng Xiao, Zhuo Diao

School of Artificial Intelligence, Chongqing University of Technology, Chongqing, 400054, China

ARTICLE INFO

Keywords:
Heterogeneous graph neural network
Flexible job shop scheduling
Deep reinforcement learning
Meta relation

ABSTRACT

The flexible job-shop scheduling problem is a critical challenge in the smart manufacturing industry. Recent methods that combine graph neural networks with deep reinforcement learning have significantly improved scheduling performance by capturing the complex features of jobs and machines. However, existing approaches still exhibit notable limitations: they often neglect the semantic relationships between nodes and fail to fully account for the heterogeneous characteristics of the relations. To address these issues, this study proposes a deep reinforcement learning approach based on a meta-relation-based heterogeneous graph neural network for solving the flexible job-shop scheduling problem. The proposed method explicitly defines distinct meta-relations to encode semantic information among different nodes and employs specially designed graph embedding techniques to capture the heterogeneity of relational structures. Experimental results on publicly available benchmark instances demonstrate that the proposed approach outperforms traditional heuristic algorithms and several state-of-the-art deep reinforcement learning models. Moreover, although the model is trained on small-scale scheduling problems, it exhibits strong generalization capability when applied to large-scale instances.

1. Introduction

Modern smart factories must quickly adapt to uncertainties such as fluctuations in market demand, machine failures, variations in raw material supply, and absenteeism of workers. Traditional production scheduling methods often struggle to effectively address these challenges. The flexible job-shop scheduling problem (FJSP) offers a solution by allowing tasks to be flexibly allocated across multiple machines and production units, thereby enhancing the adaptability of the system. This flexibility enables the production process to adjust swiftly, ensuring the efficient execution of production plans even under uncertain conditions. Furthermore, by enabling dynamic rescheduling and optimizing resource allocation, smart factories can significantly improve equipment utilization, reduce energy consumption, and strengthen the resilience of production systems against operational uncertainties (Liang, Fu, & Gao, 2024). Consequently, FJSP has become a major focus of research in both industry and academia. Unlike the job shop scheduling problem (JSP), FJSP allows operations to be performed on different machines, with machines capable of handling different operations, thus increasing its complexity and making it more relevant for real-world applications (Kacem, Hammadi, & Borne, 2002).

Recently, machine learning has been widely applied to solving combinatorial optimization problems including FJSPs. The methods used to solve FJSP mainly include exact algorithms, heuristic algorithms, metaheuristic algorithms, and deep reinforcement learning (DRL) algorithms. The exact algorithms include integer programming, dynamic programming, and branch-and-bound methods. Integer programming formulates FJSP as an optimization problem, dynamic programming solves it by breaking it into sub-problems, and branch-and-bound explores the solution space systematically while pruning suboptimal branches. However, as the size of the FJSP problem increases, the solving time for exact algorithms grows exponentially, making them difficult to apply to large-scale problems. Rule-based heuristic algorithms, such as priority dispatching rules (PDRs), assign a priority to each job and schedule jobs in order of priority, while also selecting appropriate machines to execute the tasks. These algorithms can find feasible or near-optimal solutions in a short amount of time, but designing effective priority rules requires extensive domain knowledge, and they often perform well only in specific scenarios. Metaheuristic algorithms, such as Genetic Algorithms (Sun et al., 2023; Zhang, Hu, Sun, & Zhang, 2020b), Ant Colony Optimization (Chen, Xie, Ma, Chen, & Wang, 2024), Simulated Annealing (Lim, Wong, & Chin, 2023), and Particle Swarm Optimization (Xu, Zhang, Yang, & Wang, 2024; Zhang, Zhu, & Tang, 2020c), combine local search and global search capabilities. They are more generalizable and have a stronger global search ability, making it possible to find global optimal solutions or high-quality approximations. However, the

* Corresponding author.
E-mail addresses: zhangyz@stu.cqut.edu.cn (Y. Zhang), dongsd@cqut.edu.cn (S. Dong), 417198053@stu.cqut.edu.cn (Z. Yuan), 52232313130@stu.cqut.edu.cn (T. Wen), xiaojf2000@stu.cqut.edu.cn (J. Xiao), dz2280904355@stu.cqut.edu.cn (Z. Diao).

https://doi.org/10.1016/j.eswa.2025.128411
Received 29 December 2024; Received in revised form 10 May 2025; Accepted 29 May 2025
Available online 11 June 2025

implementation complexity of these algorithms is relatively high, and they usually require parameter tuning and consume more computational resources.

DRL combines the strengths of deep neural networks and reinforcement learning. An agent interacts with the environment, using deep neural networks to process complex state and action spaces, and reinforcement learning algorithms to optimize scheduling policies. In FJSP, DRL shows significant advantages due to its powerful ability to handle complex problems and its adaptive optimization capabilities. In recent years, several studies have attempted to combine DRL with Graph Neural Network (GNN) to address FJSP. For instance, DRL has been integrated with disjunctive graphs, heterogeneous graphs, and meta-path-based heterogeneous graphs, achieving promising results. However, disjunctive graph methods fail to account for the heterogeneity between jobs and machines, heterogeneous graph-based methods struggle to capture global information effectively, and meta-path-based heterogeneous graphs face issues such as path redundancy and low computational efficiency. These methods overlook potentially valuable or latent information, such as the semantic details of different nodes and the heterogeneous characteristics between various types of relationships.

To overcome these limitations, we introduce an innovative graph neural network architecture based on meta-relations. By defining distinct meta-relations to capture semantic connections between nodes and applying tailored graph convolution methods for each relation type, this approach effectively captures their heterogeneous characteristics. This enhances the model's feature extraction capabilities, thereby improving its overall performance. To effectively address the challenges posed by complex FJSP using DRL, key issues need to be resolved: how to efficiently assign each operation to the most suitable machine; how to leverage deep neural networks to extract meaningful global feature information; and how to ensure that the scheduling policy, once trained, exhibits strong generalization capabilities across different scenarios. In light of these challenges, an end-to-end DRL approach is introduced to effectively address and solve the FJSP. First, a Markov Decision Process (MDP) is utilized to model machine and action selection. Next, a heterogeneous graph neural network based on meta-relations is designed to extract global feature information. Finally, during training, instances are generated randomly within a specified range, ensuring that the strategy remains adaptable to various problem scenarios without limitations. The main contributions of this paper can be summarized as follows:

- A novel size-agnostic end-to-end DRL framework is proposed to solve the FJSP, leveraging heterogeneous graph neural networks and meta-relations to capture complex semantic relationships between jobs and machines.
- A semantic information encoding method based on meta-relations among jobs and between jobs and machines is proposed, employing specially designed graph embedding techniques to capture the heterogeneous characteristics of these relations and enable high-quality scheduling decisions.
- A comprehensive evaluation on three public datasets demonstrates superior performance compared to state-of-the-art models and PDRs. More importantly, its ability to generalize from small-scale training instances to large-scale problems highlights its robustness and adaptability in diverse FJSP scenarios.

The remainder of the paper is structured as follows: Section 2 reviews the related work, Section 3 outlines the preliminary results, Section 4 details the proposed method to address the FJSP, Section 5 discusses the experimental results and their analysis, Section 6 concludes the paper.

2. Related work

In this section, we briefly review existing methods for solving FJSPs, including exact algorithms, heuristic and metaheuristic approaches, and deep reinforcement learning combined with graph neural networks.

2.1. Exact algorithms for solving FJSP

To obtain exact solutions, many researchers focus on studying exact algorithms to solve small-scale FJSP instances (Desaulniers, Langevin, Riopel, & Villeneuve, 2003; Lacomme, Larabi, & Tchernev, 2013; Nishi & Tanaka, 2012). Fattahi, Saidi Mehrabad, and Jolai (2007) were the first to propose a machine-location-based mixed integer linear programming (MILP) model to solve the FJSP problem. Later, Özgüven, Özbakır, and Yavuz (2010) designed a process-sequence-based MILP model, which improved both the solution efficiency and quality. Soto et al. (2020) further developed a parallel branch-and-bound method, which significantly improved the efficiency of the solution for FJSP by utilizing a parallel priority queue to manage branch sub-problems. These models can typically find exact solutions for small-scale FJSP instances. However, because of their high complexity, the solution time increases exponentially with the size of the problem, making them unsuitable for solving large-scale FJSP instances.

2.2. Heuristic methods for solving FJSP

In recent years, heuristic methods have been successfully applied to solve the FJSP. Tay and Ho (2008) addressed multi-objective FJSP using composite dispatching rules evolved through genetic programming, achieving good results on the validation set. Teymourifar, Ozturk, Ozturk, and Bahadir (2020) combined genetic programming with a simulation model, taking into account limited buffer conditions, to extract scheduling rules for solving dynamic job scheduling problems. The experimental results outperformed the classical scheduling rules. Shahgholi Zadeh, Katebi, and Doniavi (2019) designed a heuristic model to minimize the makespan of dynamic FJSP and also validated the effectiveness of the proposed approach. Although heuristic algorithms can generate high-quality solutions for FJSP in relatively short time, their effectiveness largely depends on the design of the heuristic rules. The design of such rules often requires extensive domain expertise, and as a result, the same heuristic rules may perform unsatisfactorily when applied to different problems or scenarios.

2.3. Metaheuristic methods for solving FJSP

Metaheuristic algorithms, such as genetic algorithms, tabu search, and migratory bird optimization, have achieved notable success in solving FJSPs. Xie, Li, Gao, and Gui (2023) combined the global search capability of genetic algorithms with the local search ability of tabu search, proposing a hybrid genetic tabu search algorithm that efficiently solves distributed flexible job-shop scheduling problems. Wei, He, Guo, and Hu (2023) developed a multi-objective migratory bird optimization algorithm based on game theory for dynamic flexible job-shop scheduling problems with machine failures, enhancing fairness between productivity and stability. Zhu et al. (2024) proposed a reformative memetic algorithm to address order cancellations and optimize total energy consumption in dynamic flexible job-shop environments. Chen et al. (2025) introduce a novel multipopulation-based evolutionary multi-task optimization framework with a dynamic transfer ratio to address the dynamic flexible job shop scheduling problem with additional constraints, demonstrating an improvement in scheduling efficiency compared to existing methods. Recent surveys, such as Fu, Wang, Gao, and Huang (2024), provide a comprehensive overview of ensemble meta-heuristics and reinforcement learning techniques applied to manufacturing scheduling problems, highlighting the growing importance of hybrid and learning-enhanced approaches for tackling increasingly complex and dynamic scheduling scenarios. Although metaheuristic methods demonstrate strong capabilities in exploring large solution spaces and delivering high-quality solutions, their performance often depends heavily on parameter tuning, exhibits slow convergence rates, and struggles to generalize across diverse scheduling environments, which limits their scalability and robustness in practical applications.

2.4. DRL methods for solving FJSP

Due to the strong adaptive decision-making capabilities of reinforcement learning and the powerful feature extraction abilities of deep learning, DRL provides a new direction to address large-scale and complex FJSP. Luo, Zhang, and Fan (2021) developed a two-hierarchy deep Q-network approach aimed at optimizing the total weighted tardiness and average machine utilization for multi-objective dynamic FJSP. Yue et al. (2024) proposed a two-stage double deep Q-network algorithm to handle dynamic FJSP with new job insertions and random machine failures, aiming to improve training speed and solution quality. However, these methods may face issues such as low sample efficiency and long training times when dealing with FJSPs that have complex constraints and multiple objectives.

To enhance the performance of DRL models, researchers have integrated disjunctive graphs with DRL, utilizing the complex structural information extracted by GNN to effectively capture the dependencies between operations and machines, thus optimizing scheduling policies. Han and Yang (2021) proposed an end-to-end DRL framework based on a 3D disjunctive graph, and experiments have shown that this method surpasses traditional heuristic rules. Zhang et al. (2024a) used disjunctive graphs to represent the features of the FJSP states and employed a graph attention network to extract global state information and a Transformer encoder to capture competitive relationships between machines, also achieving favorable results. However, the disjunctive graph method focuses primarily on the operating nodes, neglecting the representation of the features of the machine nodes, which may lead to inefficient resource allocation and affect the optimality of scheduling strategies.

To further improve the accuracy and comprehensiveness of state representation, researchers have introduced heterogeneous graphs to represent both operation and machine nodes, addressing their complex relationships. Song, Chen, Li, and Cao (2022) combined heterogeneous graphs with DRL to propose an end-to-end method using a graph attention network to extract feature information and trained the model using the Proximal Policy Optimization (PPO) algorithm to solve FJSP. Given the limited ability of graph attention networks (GAT) (Velickovic et al., 2017) to handle different types of nodes and edges. Wang, Wang, Sun, Deng, and Chen (2023) designed a dual attention network to enhance the feature extraction capabilities of the model, thus improving the quality of the decision-making of scheduling strategies. Li, Li, and Xu (2025) combines a hybrid heterogeneous graph neural network with principal component analysis (PCA) to solve the distributed flexible job shop scheduling problem using DRL. Nevertheless, GAT still faces limitations in processing global information in heterogeneous graphs, struggling to effectively capture complex global relationships and long-distance dependencies within the graph. In light of this, Wan, Fu, Li, and Li (2024) combined a meta-path-based heterogeneous graph neural network with an end-to-end DRL method, enhancing the capability to capture global features of heterogeneous graphs as well as improving policy learning efficiency. However, methods based on meta-paths still contend with challenges such as low computational efficiency and path redundancy.

Previous studies have made significant progress in applying graph-based DRL to FJSP. However, these methods typically model heterogeneity by distinguishing between node types and edge types. Each edge represents a basic interaction between two nodes, with the semantic meaning of these interactions implicitly encoded through the edge type, without explicitly differentiating diverse relational patterns. This approach leads to limitations in semantic modeling, especially when it comes to capturing the fine-grained semantics required in complex decision-making problems, or encountering efficiency issues due to path redundancy in meta-path-based heterogeneous graphs. In recent years, meta-relation methods have achieved significant success in capturing heterogeneous graph features by directly modeling the interactions between node types and edge types (Hu, Dong, Wang, & Sun, 2020). Meta-relations are defined as a triplet $\langle \tau(s), \phi(e), \tau(t) \rangle$, representing the source node type, edge type, and target node type, respectively. By using three interaction matrices, most weights can be shared, enabling better utilization of the heterogeneous graph's properties and capturing both common and specific patterns of different relationships. Unlike customized meta-path-based methods, meta-relations focus on local interactions between nodes and relations, avoiding the complex operations of path enumeration and aggregation required in meta-path approaches. This effectively addresses the path redundancy problem present in meta-paths, improving computational efficiency. Due to its potential, our work not only adopts a meta-relation-based framework but also extends it by assigning distinct message-passing functions to each meta-relation, enabling more fine-grained modeling. To the best of our knowledge, no meta-relation-based method has been applied to the FSJP. This work aims to fill this gap by introducing a novel meta-relation-enhanced DRL approach.

3. Preliminaries

3.1. FJSP descriptions

FJSP is a classic combinatorial optimization problem, FJSP consists of n jobs $J = J_1, \ldots, J_i, \ldots, J_n$ and m machines $M = M_1, \ldots, M_k, \ldots, M_m$. Each job J_i has multiple operations O. Each operation O_{ij} can be processed on any available machine, and different machines may have different processing times. Operations must be performed in sequence,where O_{ij} can only start after $O_{i,j-1}$ is completed. Each machine can process only one operation at a time, and it cannot be interrupted. The objective of FJSP is to select a machine and determine the sequence for each operation to achieve the goal of minimizing the maximum completion time (makespan). As shown in Fig. 1a, this is an instance containing three jobs and three machines. Each operation of a job can be executed on multiple machines, and the execution time varies across different machines. Fig. 1b presents a scheduling solution for this instance.

3.2. Heterogeneous graph

Definition 1. Heterogeneous Graph: A heterogeneous graph is characterized as a directed graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R})$ where each vertex $v \in \mathcal{V}$ and each edge $e \in \mathcal{E}$ are linked to their respective type mapping functions $\tau(v) : V \to \mathcal{A}$ and $\phi(e) : E \to \mathcal{R}$.

Meta Relation. For an edge $e = (s, t)$ connecting a source node s to a target node t, its meta-relation is represented as $\langle \tau(s), \phi(e), \tau(t) \rangle$, where $\tau(s)$ and $\tau(t)$ denote the types of source and target nodes, respectively, and $\phi(e)$ indicates the type of the edge. The inverse relation $\phi(e)^{-1}$ naturally represents the reverse of $\phi(e)$. The classical meta-path paradigm, as described in the literature (Sun, Han, Yan, Yu, & Wu, 2011; Sun et al., 2013), is defined as a sequence of such meta-relations, enabling structured, multi-step connections across different node and edge types within a heterogeneous graph.

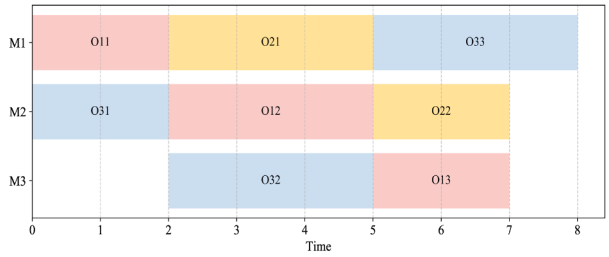
3.3. Heterogeneous graph representation of FJSP

In FJSP, the constraints on machines are more flexible, complicating the use of a disjunctive graph for representation. As a result, a heterogeneous graph structure (Song et al., 2022) $\mathcal{H} = (\mathcal{O}, \mathcal{M}, \mathcal{C}, \mathcal{E})$ is used to represent the FJSP, as shown in Fig. 2. And $\mathcal{O}$ denotes the set of operation nodes, $\mathcal{M}$ represents the set of machine nodes, and $\mathcal{C}$ is the set of conjunctive arcs, composed of directed arcs that form n paths from Start to End, each path representing the processing sequence of job J_i . $\mathcal{E}$ is the set of edges connecting machine nodes and operation nodes, indicating that operation O_{ij} can be processed on machine M_k.

Employing a heterogeneous graph to represent the FJSP offers significant advantages. Firstly, the heterogeneous graph preserves the heterogeneity between machines and operations, effectively managing multiple types of nodes. Secondly, the inclusion of rich feature information in machine nodes facilitates the capture of complex relationships between machines and operations.

Job	Operation	M_1	M_2	M_3
Job_1	O_{11}	2	6	
	O_{12}		3	4
	O_{13}	3		2
Job_2	O_{21}	3	5	
	O_{22}		2	4
Job_3	O_{31}		2	4
	O_{32}	4		3
	O_{33}	3	4	

(a) A 3×3 FJSP instance.



(b) One of the best schedules of the 3×3 FJSP instance.

Fig. 1. An illustrative example depicting a FJSP instance along with its corresponding feasible solution.

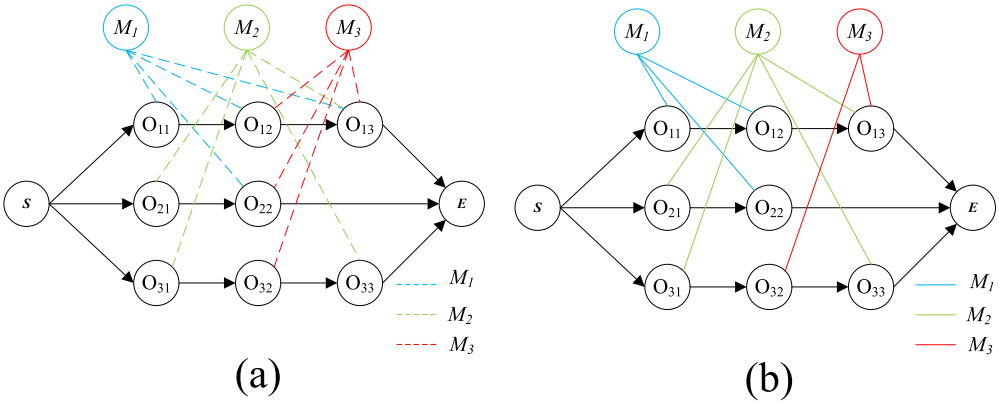


Fig. 2. Heterogeneous graph for FJSP. (a) FJSP instance. (b) Viable solution.

4. Proposed method

The detailed workflow of the proposed method is illustrated in Fig. 3. First, the system initializes the scheduling state, defines the initial configuration of jobs and machines, and represents the relationships between them using a heterogeneous graph. Next, MRHGNN is employed to perform feature embedding on the heterogeneous graph, extracting high-dimensional feature information of the jobs and machines. Then, based on the graph embedding features, the policy network generates a probability distribution for all valid operation-machine pairs and samples one operation-machine pair for scheduling. Finally, the selected operation-machine pair is executed, and the system interacts with the environment. A reward is generated based on the current scheduling performance, and the system updates to the new scheduling state. This process iterates continuously until all operations are scheduled.

4.1. Markov decision process formulation

The scheduling process is described as: At each decision step t (such as time 0 or after the completion of an operation), the agent observes the current state of the system s_t and makes a decision a_t, assigning an unscheduled ready operation to a compatible and idle machine. The operation then begins execution at the current time $T(t)$. The environment provides a reward r_t based on the operation-machine allocation and its impact on the makespan, before transitioning to the next decision step $t+1$. This iterative process continues, with the agent dynamically assigning compatible operation-machine pairs until all operations are scheduled, ultimately generating the complete solution for the FJSP. The definition of the corresponding MDP is as follows.

State: At step t, the state s_t encompasses the status information of both operations and machines. Each operation node is described by a set of four distinct features: $[SF(O_{ij}), PT(O_{ij}), WT(O_{ij}), RT(O_{ij})]$. Here, $SF(O_{ij})$ denotes the state flag of operation O_{ij}, represented using a one-hot encoding scheme to differentiate between the various states of O_{ij}. Specifically, the states are encoded as [1, 0, 0, 0] for completed, [0, 1, 0, 0] for in process, [0, 0, 1, 0] for ready, and [0, 0, 0, 1] for not ready. $PT(O_{ij})$ represents the processing time of O_{ij}. If O_{ij} has been processed on machine M_k, its processing time is $T = P_{ijk}$; otherwise, the processing time is estimated as $T = \frac{1}{|M_{ij}|}\sum_{M_k \in M_{ij}} p_{ijk}$. $WT(O_{ij})$ refers to the waiting time, which is the difference between the current time and the time when O_{ij} was first ready. $RT(O_{ij})$ represents the remaining time, defined as the sum of the processing times for all unfinished operations on the same job. Each machine node is characterized by three features: $[SF(M_k), RT(M_k), IT(M_k)]$. $SF(M_k)$ is the state flag of the machine M_k, also represented using a one-hot vector to indicate whether the machine is in use or idle. The values [1, 0] and [0, 1] correspond to the machine being active and idle, respectively. $RT(M_k)$ denotes the remaining processing time of machine M_k, calculated as the difference between the completion time of the operation currently being processed by the machine and the current time. If the machine is idle, $RT(M_k)$ is set to zero. $IT(M_k)$ represents the idle time of the machine M_k, defined as the difference between the current time and the time when the machine last became idle.

Action: The set A_t comprises all eligible pairs of operation-machine. At each decision step t, one such pair is selected for scheduling. As operations are incrementally scheduled, the action space diminishes progressively until the scheduling of all operations is completed.

Transition: Upon executing action a_t in the current state s_t, the environment updates the collections of operations and machines, as well as the action space, enabling a transition to the subsequent state s_{t+1}. The article notes that after the execution of the action a_t, the non-relevant edges are removed from the heterogeneous graph, thus differentiating two distinct states.

Reward: The primary objective of this study is to minimize the makespan, indicated as $C_{\max}$. The reward r_t is formulated as the difference in makespan during the transition from the current state s_t to

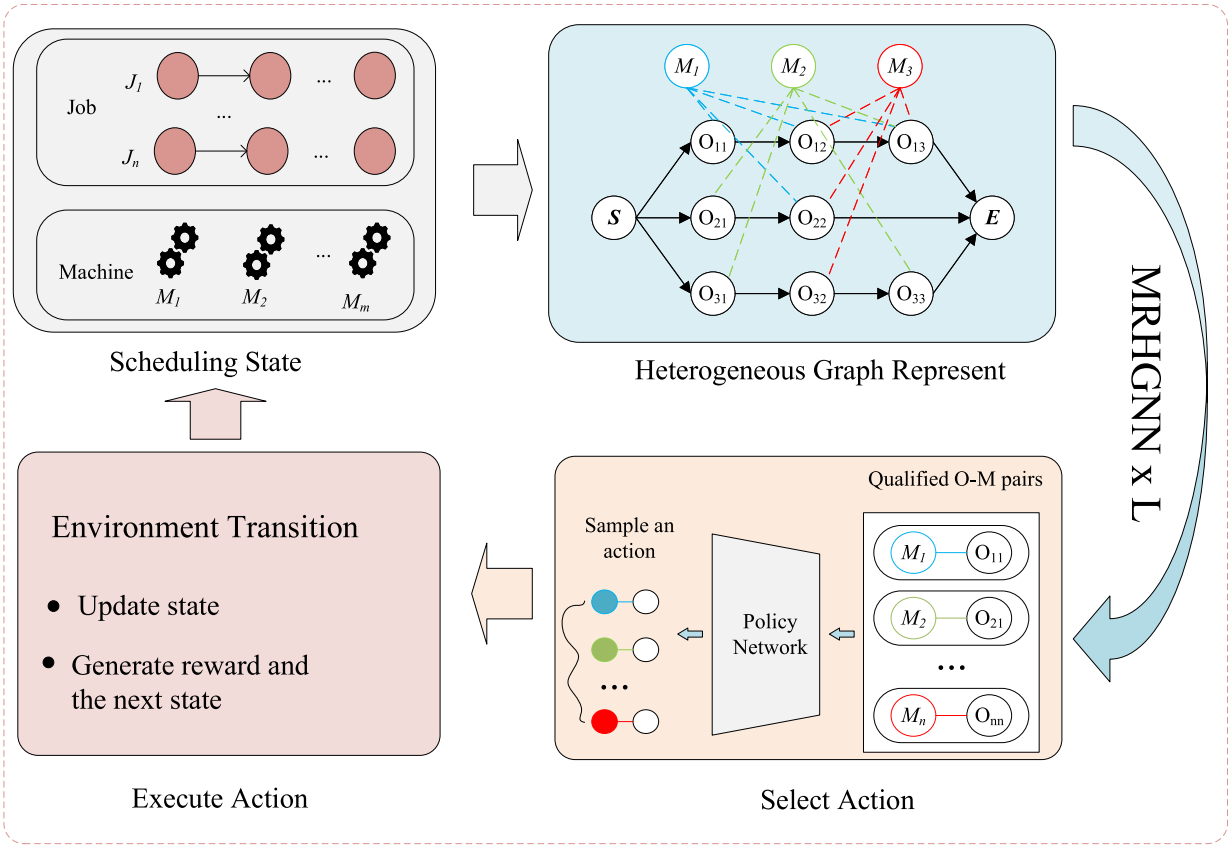


Fig. 3. Overview of the proposed method.

the subsequent state s_{t+1}, defined as $r_t(s_t, a_t, s_{t+1}) = C_{\max}(s_t) - C_{\max}(s_{t+1})$. With the discount factor γ set to 1, the cumulative reward G is calculated as $G = \sum_{t=0}^{T} r_t = C_{\max}(s_0) - C_{\max}$, where $C_{\max}(s_0)$ is a constant. Consequently, maximizing G effectively corresponds to minimizing $C_{\max}$.

Policy: This paper employs a stochastic policy $\pi(a_t|s_t)$, whose distribution is generated by a REINFORCE (Sutton & Barto, 2018) network with trainable parameters θ. Given a state s_t, this distribution returns the probability of selecting each action $a_t \in A(t)$.

4.2. Meta-relation-based HGNN

Building on previous works, this paper utilizes a heterogeneous graph representation to address the FJSP, introducing the MRHGNN architecture based on meta-relations. Meta-relations directly model the diversity and complexity between operation and machine nodes, simplifying relation modeling, reducing graph complexity, and minimizing reliance on manual design while effectively capturing semantic information. Furthermore, distinct graph convolution methods are applied to different types of meta-relations, allowing for better capture of heterogeneous characteristics across various relationships. Based on experimental analysis, this study adopts GAT and graph transform (Shi et al., 2021) as feature extraction methods for different meta-relations.

Fig. 4 shows the overall framework of the MRHGNN, which is based on meta-relations. Firstly, the heterogeneous meta-relation-based graph representation for FJSP includes four types of meta-relations: machine to machine ($M \to M$), operation processedby machine ($O \to M$), machine processes operation ($M \to O$), and operation to operation ($O \to O$). The first three relations employ GAT to capture the importance of each source node to the target node via attention mechanisms (Vaswani et al., 2017). For the final relation ($O \to O$), a graph transform is used to extract the feature information from the source operation nodes and apply it to the target operation nodes. Next, the outputs from these two models are aggregated by type, and an MLP produces the final feature embeddings, completing the feature update for both machine and operation nodes.In particular, to enhance the self-representative features of machine nodes (M_k), an additional meta-relation ($M \to M$) is included, even though it is absent in the original graph. As operation nodes inherently contain self-information within the Graph Transformer calculations, no extra self-loop is required. The feature embedding is updated as follows.

1) ($M \to M$) meta-relation feature aggregation:

To further enhance the self-representative features of the machine node M_k, attention coefficients are computed for neighboring machine nodes as well:

$$e_{kk} = \text{LeakyReLU}\left(b^T[W^M h_{M_k} \| W^M h_{M_k}]\right) \tag{1}$$

In this case, $b \in \mathbb{R}^{2d}$ is the learnable attention weight vector, the attention scores e_{kk} are normalized using the softmax function to obtain the attention coefficients α_{kk}, and the aggregation of features from neighboring machine nodes is given by:

$$h_{M_k}^{(M)} = \alpha_{kk} \cdot W^M h_{M_k} \tag{2}$$

2) ($O \to M$) meta-relation feature aggregation:

For each node in the machine M_k, the attention coefficients α_{kj} are calculated for the source operation nodes O_{ij} connected to it. This coefficient measures the influence of each operation node on the target machine node:

$$e_{kj} = \text{LeakyReLU}\left(a^T[W^O h_{O_{ij}} \| W^M h_{M_k}]\right) \tag{3}$$

Here, $h_{O_{ij}}$ and h_{M_k} represent the feature vectors of the operation and machine nodes, respectively, while W^O and W^M are the transformation matrices for their features.Then, the attention scores e_{kj} are normalized using the softmax function to obtain the attention coefficients α_{kj}. The

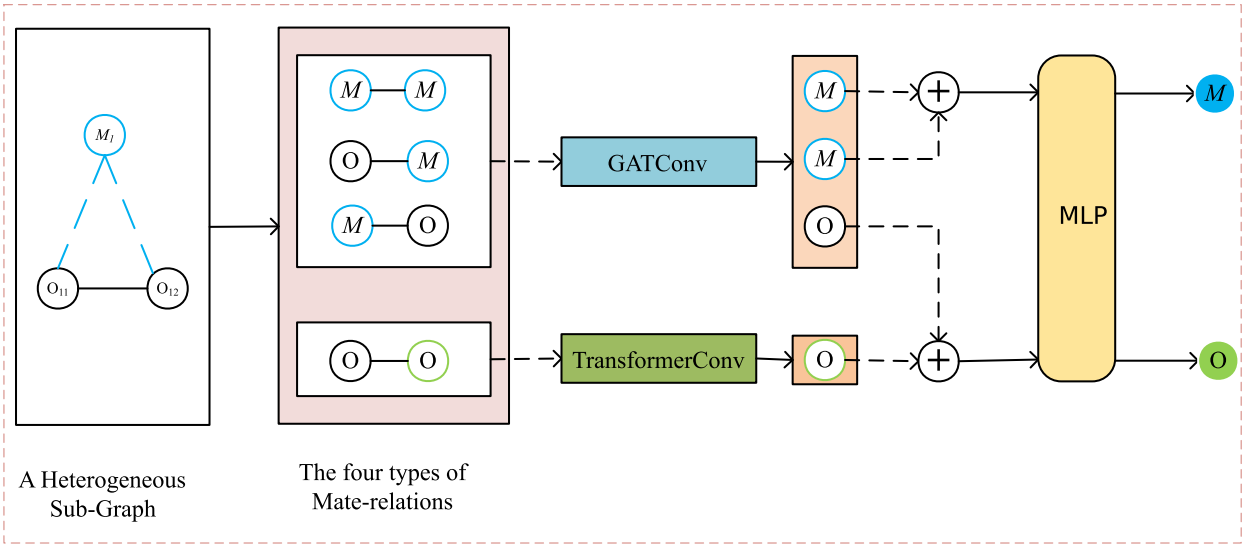


Fig. 4. The architecture of MRHGNN.

features of the source operation nodes are aggregated to the target machine node using the attention coefficients:

$$h_{M_k}^{(O)} = \sum_{j \in \mathcal{N}(M_k)} \alpha_{kj} \cdot W^O h_{O_{ij}} \tag{4}$$

Here, $\mathcal{N}(M_k)$ denotes the set of operation nodes connected to machine node M_k.

3) $(M \rightarrow O)$ meta-relation feature aggregation:

For each node of the source machine M_k connected to the operation node O_{ij}, the Graph Attention Network (GAT) computes an attention coefficient to measure the influence of M_k on O_{ij}. The attention coefficient $\alpha_{ij,k}$ is calculated as follows:

$$e_{ij,k} = \text{LeakyReLU}\left(a^T [W^O h_{O_{ij}} \| W^M h_{M_k}]\right) \tag{5}$$

where $h_{O_{ij}}$ and h_{M_k} represent the features of the operation and machine nodes, $W^O \in \mathbb{R}^{d \times d^O}$, $W^M \in \mathbb{R}^{d \times d^M}$ is a feature transformation matrix, and $a \in \mathbb{R}^{2d}$ is a learnable attention weight vector. Subsequently, the attention scores e_{ijk} are normalized using the softmax function to obtain the attention coefficients α_{ijk}. The features of the source machine nodes are then aggregated to O_{ij} through a weighted sum:

$$h_{O_{ij}}^{(M)} = \sum_{k \in \mathcal{N}(O_{ij})} \alpha_{ij,k} \cdot W^M h_{M_k} \tag{6}$$

4) $(O \rightarrow O)$ meta-relation feature aggregation:

The Graph Transformer is used to extract features from source operation nodes, enhancing O_{ij}'s information about dependencies with neighboring operation nodes. The attention coefficient between each source operation node $O_{ij'}$ and target operation node O_{ij} is computed as follows:

$$\alpha_{ij,ij'} = \text{softmax}\left(\frac{(Q_{O_{ij}})^\top K_{O_{ij'}}}{\sqrt{d}}\right) \tag{7}$$

where $Q_{O_{ij}} = W_q h_{O_{ij}}$ is the query vector, $K_{O_{ij'}} = W_k h_{O_{ij'}}$ is the key vector, and W_q and W_k are linear projection matrices. The variable d represents the dimensionality of the key and query vectors, which ensures that both $Q_{O_{ij}}$ and $K_{O_{ij'}}$ are d-dimensional. The source operation node features are then aggregated to the target operation node through a weighted sum:

$$h_{O_{ij}}^{(O)} = \sum_{O_{ij'} \in O_i} \alpha_{ij,ij'} \cdot W_v h_{O_{ij'}} \tag{8}$$

where W_v is a transformation matrix for mapping source operation node features to the target space.

The machine node feature embedding M_k is obtained by integrating its own features with the aggregated features from operation nodes, which are derived from the meta-relations $(M \rightarrow M)$ and $(O \rightarrow M)$, thereby capturing both intra-machine and machine-operation dependencies.

$$h'_{M_k} = \text{MLP}\left(h_{M_k}^{(O)} + h_{M_k}^{(M)}\right) \tag{9}$$

Here, an MLP is utilized to fuse these components, yielding the final embedding representation for the machine node.

The embedding of the feature of the operation node O_{ij} is obtained by integrating the aggregated features of both the source machine nodes and the source operation nodes, which are derived from the meta-relations $(M \rightarrow O)$ and $(O \rightarrow O)$, thus capturing the interdependencies between the machine and the operation nodes.

$$h'_{O_{ij}} = \text{MLP}\left(h_{O_{ij}}^{(M)} + h_{O_{ij}}^{(O)}\right) \tag{10}$$

Here, a MLP is used to fuse these two parts, producing the final embedding for O_{ij}. This fusion process enables the operation node embedding to incorporate both machine node information and inter-operation dependencies, enhancing the model's understanding and decision-making ability in operation scheduling tasks.

4.3. Decision making

In this section, a policy network is proposed to select the action $a_t = (O_{ij}, M_k)$ at step t. The corresponding operation and machine embedding information are fed into MLP, and the probability of the selected action is computed as follows:

$$P(a_t \mid s_t) = MLP_\phi\left[h'_{O_{ij}} \parallel h'_{M_k}\right] \tag{11}$$

where h'_{M_k} is the embedding feature of the machine node, $h'_{O_{ij}}$ is the embedding feature of the operation node, MLP_ϕ has two 256-dimensional hidden layers and the LeakRELU activation function. Then, the selection probabilities are normalized using the softmax function:

$$\pi_\phi(a_t \mid s_t) = \frac{exp(P(a_t \mid s_t))}{\sum_{b_t \in A_t} exp(P(b_t \mid s_t))} \tag{12}$$

In the training phase, a sampling strategy is implemented to randomly select actions, facilitating the exploration of various possibilities in the search for the optimal solution. Conversely, during the testing phase, a greedy strategy is employed to choose the action with the highest probability, thereby aiming to identify the optimal action.

4.4. Training

In this study, the REINFORCE algorithm with an advantage function is used to update the model parameters. The advantage function is regularized as follows:

$$A_{\pi}(S,a) = \frac{T_{\pi_b}(S,a) - T_{\pi}(S,a)}{T_{\pi_b}(S,a)} \tag{13}$$

where $A_{\pi}(S,a)$ and $T_{\pi}(S,a)$ denote the advantage and expected return associated with selecting action a in state S under policy π. In addition, $T_{\pi_b}(S,a)$ represents the expected return (or value) of selecting action a in state S when adhering to the baseline policy π_b.

A lightweight PDR is selected as the baseline, with MWKR serving as the baseline policy π_b, and previous research (Zhang et al., 2020a) has demonstrated the advantages of MWKR. The training procedure for the proposed method, as detailed in Algorithm 1, uses the REINFORCE algorithm augmented with an advantage function and an entropy-based regularization term to promote exploration. The process begins with the generation of a trajectory based on the current policy. At each iteration, the advantage function is calculated to assess the relative benefit of each action compared to a baseline policy, while the entropy term (implemented via cross-entropy loss) promotes diversity in action selection, mitigating the risk of premature convergence to suboptimal solutions. The combined objective function, which incorporates both the advantage and entropy regularization terms, is then used to update the policy parameters. This iterative process continues until the scheduling instance is successfully solved, with the learning rate adjusted every 1000 iterations to enhance stability and optimize learning performance.

Algorithm 1 Training Process for the REINFORCE Algorithm Enhanced with Normalized Advantage in FJSP.

Initialize:
  Policy parameters θ
  Learning rate α for policy
  Discount factor γ
  Entropy loss coefficient β
  Maximum number of episodes N
for $episode = 1$ to N **do**
  Generate episode: $S_1, A_1, \ldots, S_T$ using policy π_θ
  for $t = 0$ to T **do**
    Use baseline policy π_b and Calculate $T_{\pi_b}(S_t, a)$
    Calculate $T_{\pi_\theta}(S_t, a_t)$
    Calculate $A_{\pi_\theta}(S_t, a_t)$ following Eq. (13)
    Calculate entropy: $H(\pi_\theta(S_t))$
      $\theta \leftarrow \theta + \alpha\Big(\nabla_\theta \log \pi_\theta A_{\pi_\theta}(S_t, a_t)\Big)$
      $+\beta \nabla_\theta H(\pi_\theta(S_t))$
  end for
  Update learning rate every $E = 1000$ episodes
end for

5. Experiments

This section presents experimental results on public FJSP instances to validate the effectiveness of the proposed method.

5.1. Experiment settings

5.1.1. Datasets

This paper uses two types of datasets: training datasets and testing datasets. The training dataset consists of small-scale synthetic FJSP instances, where the number of jobs and machines is randomly generated between 3 and 10. The instance generation method is similar to that of Taillard (1993).

The testing datasets are divided into synthetic data and public FJSP benchmark datasets. Synthetic data are used to evaluate the performance of the model trained on small-scale instances when applied to larger-scale instances, while public benchmark datasets are used to explore the stability of the model when faced with instances of varying scales. The synthetic datasets are adapted from Wang et al. (2023), and they consist of two types: the first type (SD1) allows jobs to have a varying number of operations, and the second type (SD2) includes more machine choices per operation and more diverse processing times. In addition to the synthetic datasets, this study also uses four well-known public benchmark datasets for testing: the Brandimarte dataset, the Hurink dataset, the Behnke dataset and the Barnes dataset. The Brandimarte dataset (Brandimarte, 1993), also known as the MK dataset, contains 10 FJSP instances (mk01–mk10). The Hurink dataset (Hurink, Jurisch, & Thole, 1994) is divided into three subsets: edata, vdata, and rdata, each containing 40 FJSP instances. The Behnke dataset (Behnke & Geiger, 2012) contains 60 instances, with the smallest instance size being 10×20 and the largest instance size being 100×60. Finally, the Barnes dataset (Barnes & Chambers, 1996) includes 21 FJSP instances.

Table 1
The hyperparameter settings of the REINFORCE algorithm.

Hyperparameters	Values
Number of episodes N	3×10^6
Number of MRHGNN layers $\mathcal{L}$	3
Entropy loss coefficient η	1×10^{-2}
Learning rate α	1×10^{-4}
Discount factor γ	1
Optimizer	*Adam*

5.1.2. Configuration

The hardware platform used in this paper includes an Intel(R) Xeon(R) Gold 6226R CPU @2.90GHz and an NVIDIA Quadro RTX 5000 GPU with 16,384MB of memory. The software environment consists of Python 3.10.0 and PyTorch 2.0.1, with the default parameter settings following (Paszke, Gross, Massa, Lerer, & Chintala, 2019). The hyperparameter settings for the REINFORCE algorithm are detailed in Table 1.

5.1.3. Evaluation metric

The optimization objective of this paper is to minimize the makespan $C_{\max}$. The performance of each method is evaluated by comparing the gap between $C_{\max}$ and the best known upper bounds (UB) (Behnke & Geiger, 2012). The gap can be calculated by

$$Gap = \frac{C_{\max} - UB}{UB} \times 100\,\% \tag{14}$$

5.1.4. Baseline

A comprehensive comparison with eleven baseline methods was conducted to assess the effectiveness of the proposed method. Specifically, these baselines comprise four heuristic methods, four meta-heuristic methods, and three state-of-the-art DRL approaches, as outlined below:

1) Rule-Based Heuristic Methods (PDRs):

- MOR (Most Operations Remaining): Prioritizes jobs with the most operations left to process.
- FIFO (First In, First Out): Schedules jobs based on their arrival order.
- SPT (Shortest Processing Time): Gives priority to jobs with the shortest processing time.
- MWKR (Most Work Remaining): Prioritize jobs with the most work remaining in terms of total processing time.

2) Meta-Heuristic Methods:

- IJA (Improved Job Assignment) (Caldeira, 2019): IJA optimizes the assignment of jobs to resources, improving scheduling efficiency by enhancing job-resource matching.

Table 2
Experimental results on the large-sized synthetic data.

	Size		MOR	FIFO	SPT	MKWR	HGNN	DANIEL	ReSch	Ours	OR-Tools
SD1	30x10	C_{max}	317.43	328.24	350.07	312.93	313.04	281.49	293.04	**278.2**	
		Time(s)	1.10	1.09	1.09	1.09	2.84	2.76	3.57	4.31	274.67 (6 %)
		Gap	15.60 %	19.56 %	27.47 %	13.96 %	14.01 %	2.50 %	6.71 %	**1.31 %**	
	40x10	C_{max}	421.34	426.97	445.17	414.82	416.18	371.45	384.31	**367.47**	
		Time(s)	1.51	1.49	1.50	1.50	3.81	3.77	5.41	6.67	365.96 (3 %)
		Gap	15.16 %	16.70 %	21.66 %	13.37 %	13.75 %	1.52 %	5.03 %	**0.43 %**	
SD2	30x10	C_{max}	1555.11	1566.17	1105.99	1539.67	1543.69	**774.56**	851.11	776.3	
		Time(s)	1.12	1.11	1.10	1.10	2.93	2.75	3.25	4.07	692.26 (0 %)
		Gap	125.16 %	126.75 %	59.74 %	122.89 %	123.57 %	**11.95 %**	23.06 %	12.23 %	
	40x10	C_{max}	2052.13	2059.3	1357.16	2037.65	2032.54	962.58	1036.8	**961.84**	
		Time(s)	1.52	1.52	1.50	1.51	3.93	3.74	4.95	5.7	998.39 (0 %)
		Gap	110.21 %	110.94 %	37.74 %	108.66 %	108.12 %	−1.67 %	5.98 %	**−1.69 %**	

- RegGA (Regular Genetic Algorithm) (Rooyani & Defersha, 2019): RegGA applies a standard genetic algorithm framework that uses selection, crossover, and mutation operations to approximate optimal solutions within the solution space.
- 2SGA (Two-Stage Genetic Algorithm) (Rooyani & Defersha, 2019): 2SGA operates in two stages, initially conducting a broad search before refining promising solutions in a second fine-tuning phase.
- SLGA (Simulated Annealing and Genetic Algorithm) (Chen, Yang, Li, & Wang, 2020): SLGA combines simulated annealing and genetic algorithm techniques, leveraging the global exploration capability of simulated annealing and the optimization strengths of genetic algorithms to achieve optimal solutions.

3) DRL Methods:

- HGNN (Song et al., 2022):An end-to-end deep reinforcement learning framework employing a heterogeneous graph neural network is proposed to learn high-quality priority dispatching rules, thereby enabling efficient solution of the FJSP.
- DANIEL (Wang et al., 2023): A dual-attention network method based on reinforcement learning is proposed, which utilizes two interconnected attention modules to model the feature representations of machine nodes and operation nodes. These features are then integrated with reinforcement learning to solve FJSP.
- LMLPF (Yuan et al., 2024): This study proposes a method that leverages a lightweight MLP to extract feature information, combined with a novel state representation and action space, to address FJSP.
- ResSch (Ho et al., 2024): This method redefines the job scheduling process and introduces a residual scheduling approach to solve FJSP.

5.2. Convergence analysis for policy training

To evaluate the convergence behavior of the proposed policy, three representative benchmark datasets-MK, Hurink-rdata, Hurink-edata, and Hurink-vdata-were selected for testing. During training, the policy model was saved every 1,000 iterations, resulting in a total of 300 checkpoints over 300,000 training steps. Each checkpoint was evaluated by computing the average gap on the selected datasets, where the GAP is defined as the relative difference between the obtained makespan and the known optimal or best-known solution.

As shown in Fig. 5, the average GAP across all datasets drops rapidly in the early stage of training, indicating fast learning progress. After approximately 50,000 iterations, the performance gradually stabilizes, with only minor fluctuations observed in the later stages. Overall, the model exhibits a clear and consistent convergence trend across different datasets.

5.3. Experimental results on synthetic data

In Table 2, we present a summary of the average makespan, computation time, and gap for the PDRs and DRL methods when handling the SD1 and SD2 instances with sizes 30×10 and 40×10. These metrics are used to assess the model's ability to generalize to larger-scale problems. As indicated by the bold values, our approach significantly outperforms the PDRs method and is comparable to, or exceeds, the DRL method across the four instance sizes. For the SD1 dataset, on the 30×10 and 40×10 instances, our method achieves gaps of 1.31 % and 0.43 %, respectively, which are much lower than those of PDRs and outperform DRL. For the SD2 dataset, our method shows a clear advantage over ResSch, and is very close to DANIEL. Excitingly, on the SD2 40×10 FJSP instances, our method surpassed OR-Tools, achieving a performance improvement of 1.69 %. These results indicate that our model is capable of capturing fundamental semantic relationships between jobs, operations, and machines, such as precedence constraints, machine flexibility, and other features. These relational patterns remain consistent across scheduling instances of varying scales, allowing the model not only to learn effective scheduling strategies on smaller-scale instances but also to efficiently transfer these strategies to larger-scale instances, ensuring stability and adaptability when facing more complex problems. It is important to note that, for fairness in comparison, all three DRL methods employed a greedy strategy. Since the source code for LMPF is not publicly available, it is not included in this table. As for DANIEL, we selected the model trained on the 20×10 instances, which performs best across the four instance sizes, as the comparison baseline. For ResSch, the results were obtained using publicly available source code and the trained model.

We add the visualization of the comparison results to better demonstrate the performance of the baseline methods on the SD1 and SD2 datasets. Fig. 6 presents the summary chart, from which it is clear that the baseline methods perform significantly worse on the SD2 dataset compared to the SD1 dataset. Our method outperforms PDRs, HGNN, and ReSch in several aspects and exhibits higher stability. Compared to the DANIEL method, although our method achieves performance close to theirs, it slightly underperforms on the SD2 30x10 dataset. However, on the other three datasets, our method performs even better.

5.4. Experimental results on public benchmarks

5.4.1. Performance comparison based on average GAP

Table 3 reports the makespan average gap (%) and computation time (in seconds) for the proposed method and several baseline algorithms across six public FJSP benchmark datasets. These datasets vary in size and complexity, enabling a comprehensive evaluation of each method's generalization and robustness. To ensure a fair comparison, all DRL-based approaches adopt a greedy decoding strategy during inference. The results for DANIEL and LMLPF are taken directly from their original publications, while those for ResSch are reproduced using their publicly available source code and pre-trained models.

Among the evolutionary algorithms, IJA and RegGA generally deliver high-quality solutions. For example, IJA achieves a competitive

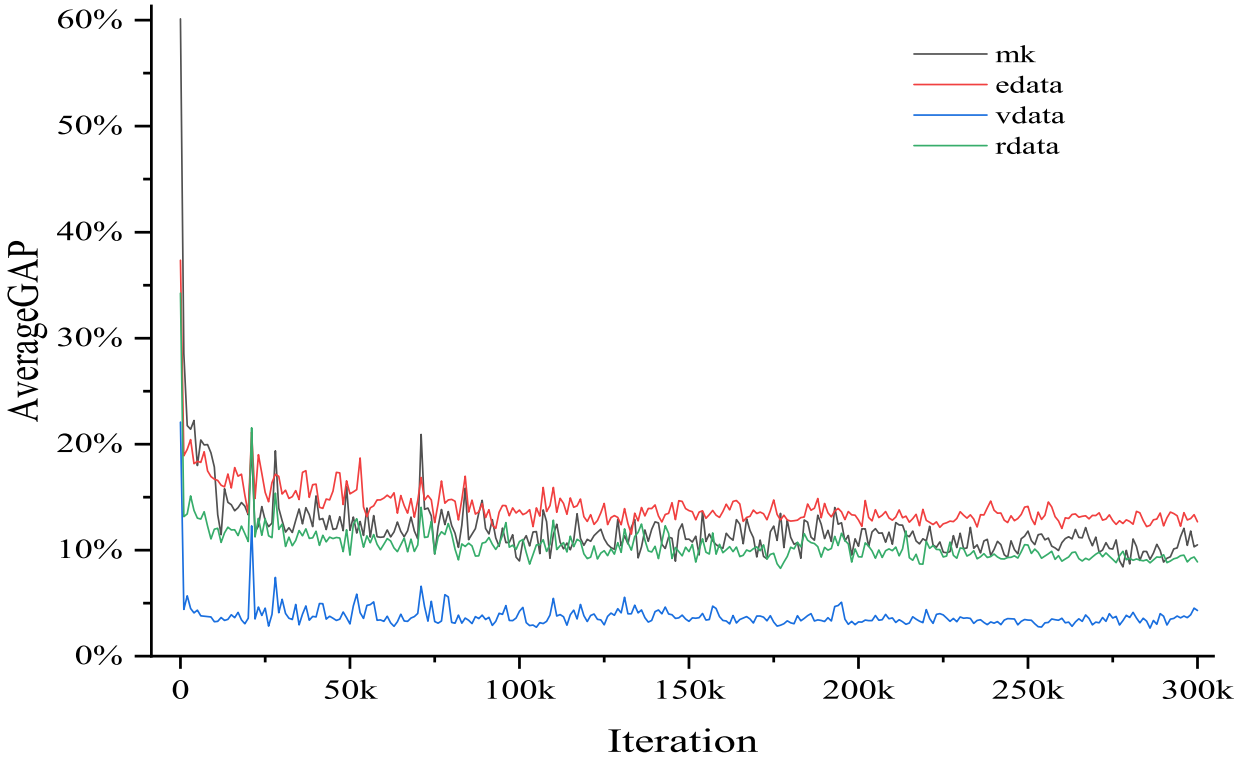


Fig. 5. Policy Convergence Evaluated by Average GAP on Benchmark Datasets.

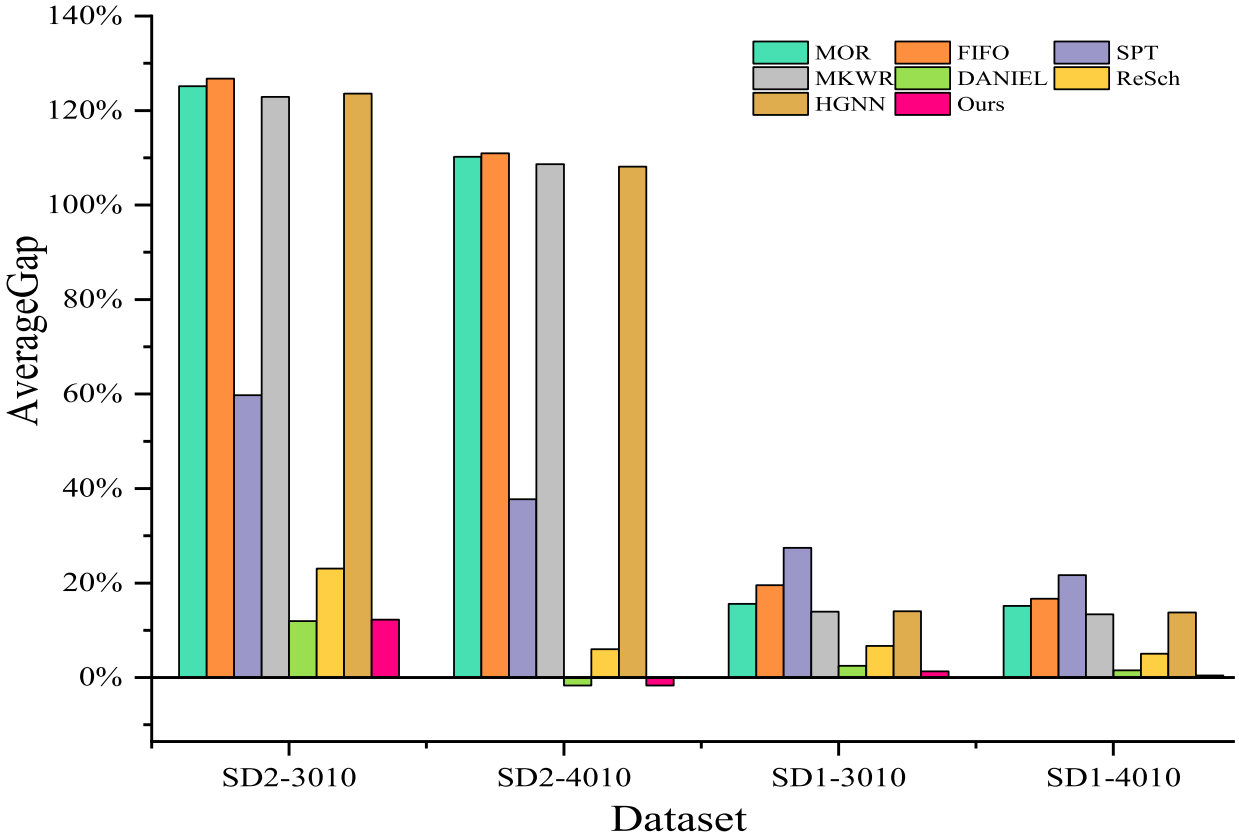


Fig. 6. Comparison chart with other DRL methods and composite PDRs.

Table 3
Comparison of average gap and average time on public datasets with various baselines.

Methods	Barnes		Brandimarte		Hurink (rdata)		Hurink (edata)		Hurink (vdata)		Behnke	
	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)	Gap	Time (s)
IJA	2.40 %	21.03	8.50 %	15.43	3.90 %	19.72	4.60 %	15.24	2.70 %	17.85	–	–
RegGA	–	–	8.39 %	280.10	–	–	–	–	3.20 %	191.40	–	–
2SGA	–	–	3.17 %	57.60	–	–	–	–	0.39 %	51.43	–	–
SLGA	–	–	6.21 %	283.28	–	–	–	–	–	–	–	–
MOR	24.84 %	0.25	28.08 %	0.48	15.07 %	0.52	19.24 %	0.52	5.68 %	0.52	44.15 %	1.06
SPT	24.49 %	0.24	44.88 %	0.48	29.47 %	0.52	26.79 %	0.51	18.20 %	0.52	2.90 %	1.01
FIFO	28.26 %	0.24	31.82 %	0.48	17.25 %	0.52	20.83 %	0.52	7.58 %	0.52	43.56 %	1.09
MWKR	20.59 %	0.26	28.91 %	0.49	13.86 %	0.52	18.60 %	0.52	4.22 %	0.52	42.27 %	1.10
HGNN	13.84 %	2.12	27.83 %	0.91	11.15 %	1.03	15.53 %	1.00	4.25 %	1.00	42.30 %	3.39
DANIEL	15.33 %	1.84	12.97 %	1.30	11.42 %	1.37	14.41 %	1.38	3.28 %	1.37	8.93 %	3.35
LMLPF	13.83 %	0.39	13.24 %	0.40	12.09 %	0.27	15.54 %	0.27	5.37 %	0.27	–	–
ResSch	15.16 %	0.68	9.81 %	0.85	9.65 %	0.63	13.23 %	0.81	3.82 %	0.98	5.40 %	4.43
Ours	**13.23 %**	1.02	**9.13 %**	1.26	**8.80 %**	1.27	**12.34 %**	1.01	**2.64 %**	1.50	**1.93 %**	4.05

gap of 2.70 % on the Hurink (vdata) dataset and 2.40 % on Barnes, outperforming most PDR and DRL-based baselines. However, this comes at the cost of significantly higher computation times-often exceeding 17 s per instance. Similarly, RegGA attains a 3.20 % gap on vdata but requires an excessive 191.40 s, limiting its applicability in time-sensitive scenarios. In contrast, PDR-based methods (e.g., MOR, SPT, FIFO, MWKR) are highly efficient, typically requiring less than one second per instance. Nevertheless, their solution quality is considerably lower, with average gaps frequently exceeding 20 %, making them unsuitable for applications that demand high scheduling precision. DRL-based baselines (HGNN, DANIEL, LMLPF, and ResSch) offer a compromise between runtime and performance, but their results are still inconsistent across datasets. For instance, DANIEL yields a 12.97 % gap on Brandimarte, while LMLPF performs poorly on Hurink (vdata), reaching a gap of 5.37 %. By comparison, the proposed method consistently achieves superior performance across all datasets. Notably, on Hurink (vdata), it achieves the best result of 2.64 %, outperforming IJA (2.70 %) with significantly lower computation time (1.50s vs. 17.85s). It also performs well on Brandimarte (9.13 %), rdata (8.80 %), and edata (12.34 %), with runtimes maintained within a practical range of 1–1.5 s. On the large-scale Behnke dataset, the proposed method achieves a notable improvement, with a gap of 1.93 %, clearly outperforming the best DRL baseline (ResSch at 5.40 %) and approaching the performance of traditional optimization algorithms. These results demonstrate that the proposed MRHGNN effectively captures high-dimensional scheduling information and generalizes well across diverse FJSP instances. It offers a compelling balance between optimization quality and computational efficiency.

To better illustrate the comparison results, Fig. 7 visualizes the makespan average gap performance across all datasets. Compared with DRL methods and composite PDR strategies, the proposed method achieves the lowest gap consistently and shows stable performance, confirming its superior optimization ability and robustness.

5.4.2. Stability analysis based on GAP distribution

To investigate the robustness and consistency of different methods, we visualize the distribution of average makespan gaps across benchmark datasets using boxplots, as shown in Fig. 8. Each subfigure corresponds to a specific dataset, allowing for a comparative analysis of performance variability.

As illustrated in Fig. 8a (Brandimarte), Fig. 8d (vdata), and Fig. 8e (Barnes), the proposed method consistently exhibits narrower interquartile ranges (IQRs) and shorter overall whiskers than the baseline methods. This indicates lower variance and higher stability in performance. Even in datasets where the relative advantage is less pronounced-such as Fig. 8b (rdata), Fig. 8c (edata), and Fig. 8f (Behnke)-our method still achieves the lowest median gap in all cases, showing reliable superiority over both PDR and DRL-based baselines. Overall, the proposed method consistently delivers high-quality solutions across problems of varying scales, further demonstrating its stability and generalization capability.

Table 4
Results of Friedman and Nemenyi post-hoc test of Ours and baselines.

Methods	Average ranking value	Final rank	Difference	Critical range value
HGNN	3.305	4	1.126	0.702
Daniel	2.987	3	0.808	
LMLPF	3.510	5	1.331	
ResSch	2.709	2	0.530	
Ours	2.179	1	0.000	

5.4.3. Statistical significance test

To comprehensively evaluate the relative performance of the proposed method, this study adopts well-established statistical techniques widely used in prior works (Fu et al., 2025; Su, Fu, Gao, Dong, & Mou, 2023; Zhang, Fu, Gao, Pan, & Huang, 2024b), including the Friedman test followed by the Nemenyi post-hoc test (Brown & Mues, 2012). These tests are conducted on the GAP results obtained from several DRL approaches, including the proposed method, across 151 public benchmark instances. The detailed statistical outcomes are summarized in Table 4. Due to the unavailability of publicly accessible data, meta-heuristic algorithms are excluded from the comparison. Furthermore, PDR-based methods are omitted to focus the analysis exclusively on learning-based strategies. In addition, as the performance data of the LMLPF method on the Behnke dataset are unavailable, this dataset is excluded from the evaluation. Instead, the comparative analysis is performed on 10 instances from the MK dataset, 21 from the Barnes dataset, and 40 instances each from the rdata, edata, and vdata subsets of the Hurink dataset, resulting in a total of 151 instances used for statistical evaluation.

For the Friedman test at a significance level of 0.05, the average ranking values across all instances of the five algorithms are presented in Table 4. The resulting p-value is 0.0000, which is smaller than the significance threshold. Hence, the differences among the five approaches are statistically significant. This study further adopts the Nemenyi post-hoc test with the same significance level to examine pairwise differences among the methods. As shown in Table 4, the proposed method ("Ours") achieves the lowest average ranking value of 2.179 and is ranked first overall. According to the Nemenyi test, the critical difference at a 0.05 significance level is 0.702. The ranking differences between "Ours" and HGNN (1.126), LMLPF (1.331), and Daniel (0.808) all exceed this threshold, indicating that "Ours" significantly outperforms these baselines. In contrast, the difference between "Ours" and ResSch is only 0.530, which is below the critical value, suggesting that the performance of these two methods is statistically indistinguishable.

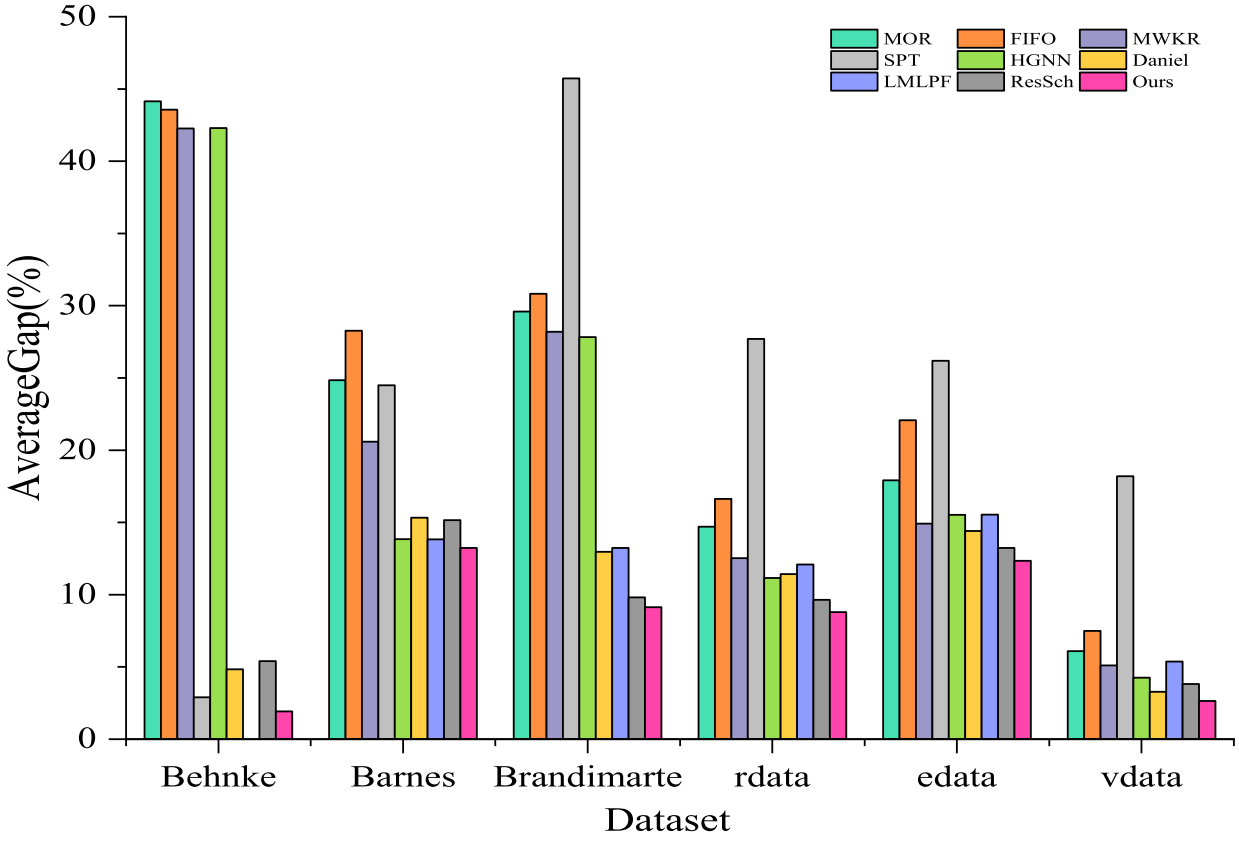
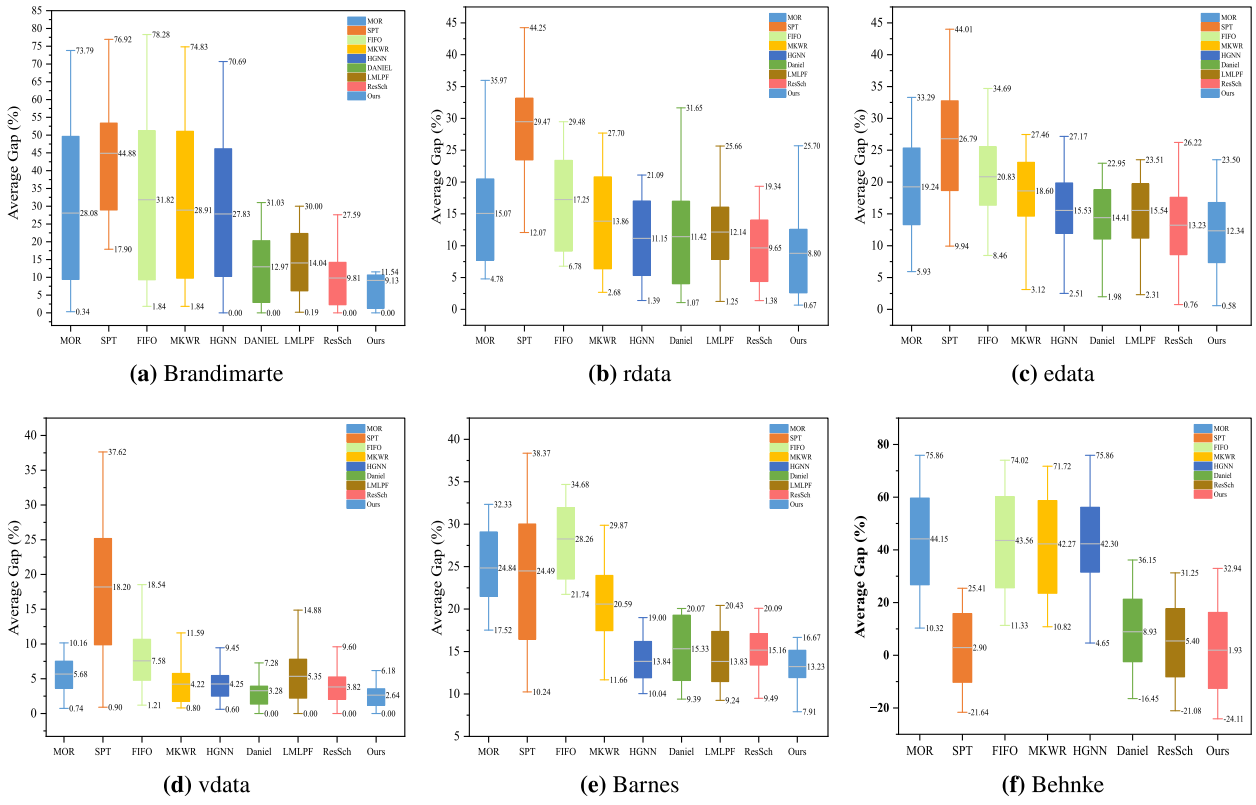


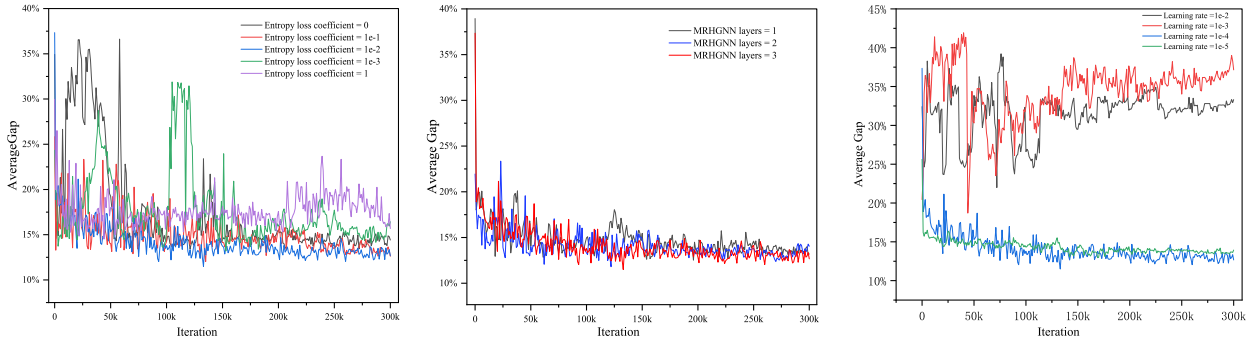
Fig. 7. Comparison chart with other DRL methods and composite PDRs.



(a) Brandimarte **(b)** rdata **(c)** edata

(d) vdata **(e)** Barnes **(f)** Behnke

Fig. 8. Relative gaps of baseline methods and ours on various datasets.



(a) Entropy loss coefficient **(b)** MRHGNN Layers **(c)** Learing rate

Fig. 9. Sensitivity analysis of three key hyperparameters.

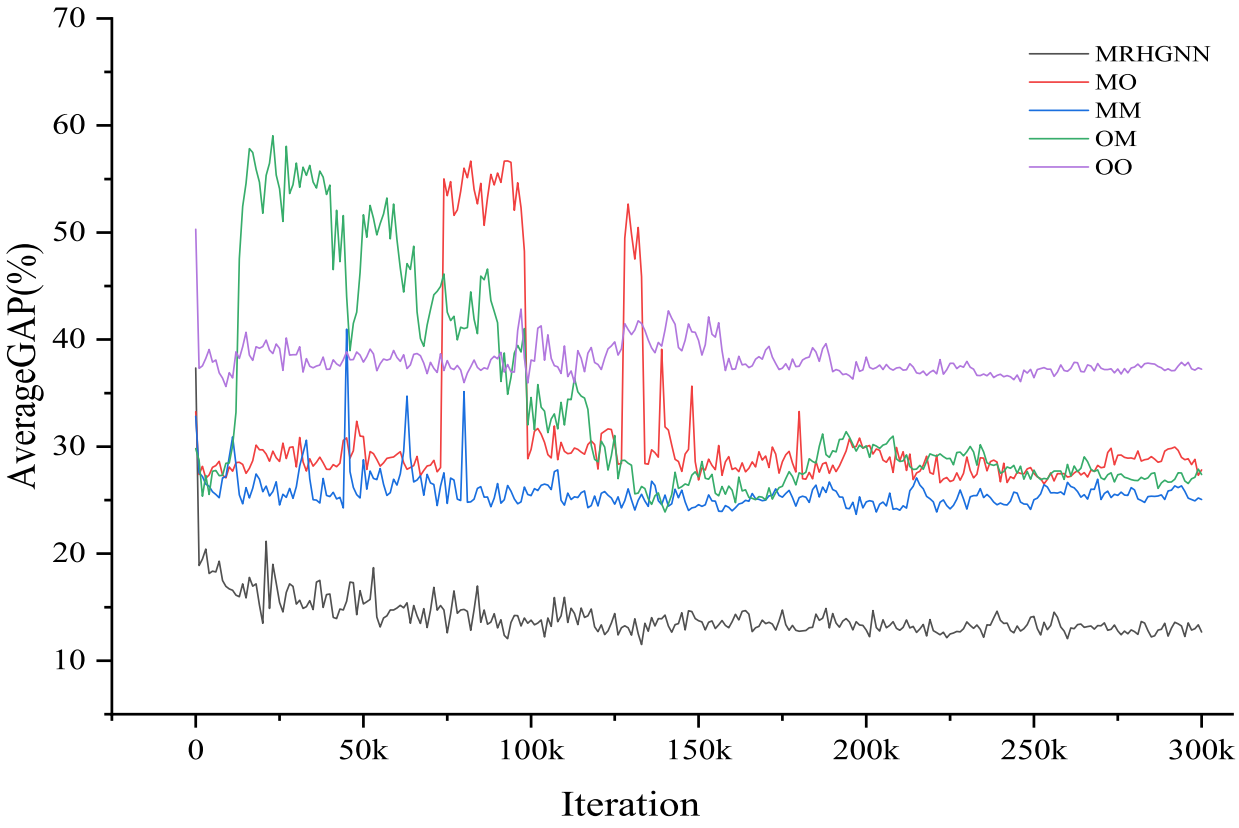


Fig. 10. Training convergence curves under different meta-relation ablation settings.

5.5. Parameter sensitivity analysis

The performance of deep reinforcement learning models is often influenced by the selection of key hyperparameters. To evaluate the robustness and stability of the proposed MRHGNN-based scheduling framework, we conducted a parameter sensitivity analysis focusing on three critical hyperparameters: the entropy loss coefficient, the number of GNN layers, and the learning rate. The results are illustrated in Fig. 9, showing the evolution of the average gap during training. As shown in Fig. 9a, setting the entropy loss coefficient to 0 leads to unstable training and poor performance due to insufficient exploration. In contrast, an excessively large value (e.g., 1.0) introduces too much randomness, hindering convergence. Entropy loss coefficients of 1e-2 and 1e-3 consistently yield lower average gaps and better stability, with 1e-2 exhibiting the best overall performance. Therefore, this value is selected to ensure a balanced exploration-exploitation trade-off. Fig. 9b compares models with 1, 2, and 3 layers in the MRHGNN architecture. The 1-layer setting underperforms due to limited expressive power. Both 2-layer and 3-layer configurations achieve similar results, but the 3-layer model shows slightly better convergence and lower final gaps. Thus, a 3-layer GNN is adopted to enhance representation capacity without significantly increasing complexity. As illustrated in Figure Fig. 9c, the learning rate plays a crucial role in the convergence process. A learning rate of 1e-2 and 1e-3 results in excessive variance and slow convergence, which hampers efficient learning. On the other hand, a learning rate of 1e-5 leads to slow learning and suboptimal performance. The learning rate of 1e-4 offers the best performance, enabling fast and stable convergence with a strong final outcome.

5.6. Ablation study on meta-relation design

To validate the necessity of the proposed meta-relation design, we conduct ablation studies by selectively removing specific meta-relations and observing their impact on the model's training stability and final performance.

As illustrated in Fig. 10, the MRHGNN model, which incorporates all meta-relations, achieves the best performance with the lowest makespan and the fastest convergence. When the **MO** ($M \rightarrow O$) is removed, the model shows noticeable instability during training and converges to a higher makespan, indicating the importance of directly modeling machine capabilities for operations. Removing the **MM** ($M \rightarrow M$) results in a slight performance drop, suggesting that inter-machine communication, while helpful, is relatively less critical than other relations. The removal of the **OM** ($O \rightarrow M$) severely degrades the training stability and leads to a significantly worse final makespan, highlighting that associating operations with their processing resources is crucial for effective decision making. Finally, when the **OO** ($O \rightarrow O$) is removed, the model exhibits the worst performance, with consistently high makespan values and poor convergence, demonstrating the vital role of capturing sequential and precedence dependencies between operations. Overall, these results confirm that all proposed meta-relations contribute meaningfully to the model's scheduling capability, with operation-to-operation and operation-machine interactions being particularly essential.

6. Conclusion

Efficient resolution of FJSP is crucial for advancing smart manufacturing systems. In this study, we proposed a novel end-to-end framework that integrates DRL with MRHGG to address the inherent complexities of FJSP. The scheduling problem was modeled as a heterogeneous graph, where multiple meta-relations were explicitly designed to capture the semantic interactions among jobs, operations, and machines. Distinct graph convolution mechanisms were employed for different meta-relations to accurately represent heterogeneous relational characteristics. The high-dimensional embeddings extracted from this process were subsequently fed into a decision network, which was trained via the REINFORCE algorithm to learn efficient scheduling strategies.

Compared to traditional PDRs and existing DRL-based methods, the proposed approach offers several notable advantages. First, by modeling rich semantic interactions through meta-relations, the MRHGG framework achieves a significantly stronger representation capability, enabling it to capture complex structural dependencies that conventional methods overlook. Second, the use of heterogeneous convolution operations for different types of relations allows the model to handle diverse relational patterns more effectively, enhancing its adaptability across varied scheduling scenarios. Third, experimental results demonstrate that our method exhibits exceptional scalability and generalization, as models trained on randomly generated small-scale instances can deliver high-quality solutions when applied to large-scale FJSP problems. These advantages collectively contribute to the superior performance of the proposed method in both solution quality and robustness. By training the model on small synthetic instances and successfully generalizing it to larger, unseen problems, we have significantly reduced the computational and data collection burdens typically associated with large-scale scheduling. Moreover, the meta-relation-driven graph modeling approach lays a solid foundation for extending to dynamic and uncertain scheduling scenarios. In future work, we plan to extend the MRHGG framework to dynamic FJSP settings and integrate predictive maintenance and real-time decision-making modules, further broadening its applicability in intelligent manufacturing systems.

CRediT authorship contribution statement

Yuzhi Zhang: Conceptualization, Methodology, Investigation, Validation, Formal analysis, Writing – original draft, Writing – review & editing; **Shidu Dong:** Supervision, Writing – review & editing; **Zhenfang Yuan:** Investigation; **Ting Wen:** Investigation; **Jianfeng Xiao:** Data curation; **Zhuo Diao:** Data curation, Validation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Barnes, J. W., & Chambers, J. B. (1996). Flexible job shop scheduling by tabu search. *Graduate program in operations and industrial engineering, The University of Texas at Austin, Technical Report Series, ORP96-09*.

Behnke, D., & Geiger, M. J. (2012). Test instances for the flexible job shop scheduling problem with work centers. Arbeitspapier/Research Paper, Helmut-Schmidt-Universität, Lehrstuhl für Betriebswirtschaftslehre, insbes. Logistik-Management.

Brandimarte, P. (1993). Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, *41*(3), 157–183.

Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, *39*(3), 3446–3453.

Caldeira, R. H., & Gnanavelbabu, A. (2019). Solving the flexible job shop scheduling problem using an improved jaya algorithm. *Computers & Industrial Engineering*, *137*, 106064.

Chen, F., Xie, W., Ma, J., Chen, J., & Wang, X. (2024). Textile flexible job-shop scheduling based on a modified ant colony optimization algorithm. *Applied Sciences*, *14*(10), 4082.

Chen, R., Yang, B., Li, S., & Wang, S. (2020). A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, *149*, 106778.

Chen, X., Li, J., Wang, Z., Chen, Q., Gao, K., & Pan, Q. (2025). Optimizing dynamic flexible job shop scheduling using an evolutionary multi-task optimization framework and genetic programming. *IEEE Transactions on Evolutionary Computation* https://doi.org/10.1109/TEVC.2025.3543770.

Desaulniers, G., Langevin, A., Riopel, D., & Villeneuve, B. (2003). Dispatching and conflict-free routing of automated guided vehicles: An exact approach. *International Journal of Flexible Manufacturing Systems*, *15*(4), 309–331.

Fattahi, P., Saidi Mehrabad, M., & Jolai, F. (2007). Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *Journal of Intelligent Manufacturing*, *18*, 331–342.

Fu, Y., Gao, K., Wang, L., Huang, M., Liang, Y.-C., & Dong, H. (2025). Scheduling stochastic distributed flexible job shops using an multi-objective evolutionary algorithm with simulation evaluation. *International Journal of Production Research*, *63*(1), 86–103.

Fu, Y., Wang, Y., Gao, K., & Huang, M. (2024). Review on ensemble meta-heuristics and reinforcement learning for manufacturing scheduling problems. *Computers and Electrical Engineering*, *120*, 109780.

Han, B., & Yang, J. (2021). A deep reinforcement learning based solution for flexible job shop scheduling problem. *International Journal of Simulation Modelling*, *20*(2), 375–386.

Ho, K.-H., Cheng, J.-Y., Wu, J.-H., Chiang, F., Chen, Y.-C., Wu, Y.-Y., & Wu, I.-C. (2024). Residual scheduling: A new reinforcement learning approach to solving job shop scheduling problem. *IEEE Access*,*12*, 14703–14718.

Hu, Z., Dong, Y., Wang, K., & Sun, Y. (2020). Heterogeneous graph transformer. In *Proceedings of the web conference 2020* (pp. 2704–2710).

Hurink, J., Jurisch, B., & Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *Operations-Research-Spektrum*, *15*, 205–215.

Kacem, I., Hammadi, S., & Borne, P. (2002). Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, *32*(1), 1–13.

Lacomme, P., Larabi, M., & Tchernev, N. (2013). Job-shop based framework for simultaneous scheduling of machines and automated guided vehicles. *International Journal of Production Economics*, *143*(1), 24–34.

Li, J., Li, J., & Xu, Y. (2025). HGNP: A PCA-based heterogeneous graph neural network for a family distributed flexible job shop. *Computers & Industrial Engineering*, *200*, 110855.

Liang, P., Fu, Y., & Gao, K. (2024). Multi-product disassembly line balancing optimization method for high disassembly profit and low energy consumption with noise pollution constraints. *Engineering Applications of Artificial Intelligence*, *130*, 107721.

Lim, K. C. W., Wong, L.-P., & Chin, J. F. (2023). Simulated-annealing-based hyper-heuristic for flexible job-shop scheduling. *Engineering Optimization*, *55*(10), 1635–1651.

Luo, S., Zhang, L., & Fan, Y. (2021). Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Computers & Industrial Engineering*, *159*, 107489.

Nishi, T., & Tanaka, Y. (2012). Petri net decomposition approach for dispatching and conflict-free routing of bidirectional automated guided vehicle systems. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans*, *42*(5), 1230–1243.

Özgüven, C., Özbakır, L., & Yavuz, Y. (2010). Mathematical models for job-shop scheduling problems with routing and process plan flexibility. *Applied Mathematical Modelling*, *34*(6), 1539–1548.

Paszke, A., Gross, S., Massa, F., Lerer, A., & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. arXiv preprint arXiv:1912.01703

Rooyani, D., & Defersha, F. M. (2019). An efficient two-stage genetic algorithm for flexible job-shop scheduling. *IFAC-PapersOnLine*, *52*(13), 2519–2524.

Shahgholi Zadeh, M., Katebi, Y., & Doniavi, A. (2019). A heuristic model for dynamic flexible job shop scheduling problem considering variable processing times. *International Journal of Production Research*, *57*(10), 3020–3035.

Shi, Y., Huang, Z., Feng, S., Zhong, H., Wang, W., & Sun, Y. (2021). Masked label prediction: Unified message passing model for semi-supervised classification. In *International joint conference on Artificial Intelligence*.

Song, W., Chen, X., Li, Q., & Cao, Z. (2022). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, *19*(2), 1600–1610.

Soto, C., Dorronsoro, B., Fraire, H., Cruz-Reyes, L., Gomez-Santillan, C., & Rangel, N. (2020). Solving the multi-objective flexible job shop scheduling problem with a novel parallel branch and bound algorithm. *Swarm and Evolutionary Computation*, *53*, 100632.

Su, J., Fu, Y., Gao, K., Dong, H., & Mou, J. (2023). Integrated scheduling problems of open shop and vehicle routing using an ensemble of group teaching optimization and simulated annealing. *Swarm and Evolutionary Computation*, *83*, 101373.

Sun, K., Zheng, D., Song, H., Cheng, Z., Lang, X., Yuan, W., & Wang, J. (2023). Hybrid genetic algorithm with variable neighborhood search for flexible job shop scheduling problem in a machining system. *Expert Systems with Applications*, *215*, 119359.

Sun, Y., Han, J., Yan, X., Yu, P. S., & Wu, T. (2011). PathSim: Meta path-based top-K similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, *4*(11), 992–1003.

Sun, Y., Norick, B., Han, J., Yan, X., Yu, P. S., & Yu, X. (2013). PathSelClus: Integrating meta-path selection with user-guided object clustering in heterogeneous information networks. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, *7*(3), 1–23.

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: an introduction. MIT Press.

Taillard, E. D. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, *64*(2), 278–285.

Tay, J. C., & Ho, N. B. (2008). Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Computers & Industrial Engineering*, *54*(3), 453–473.

Teymourifar, A., Ozturk, G., Ozturk, Z. K., & Bahadir, O. (2020). Extracting new dispatching rules for multi-objective dynamic flexible job shop scheduling with limited buffer spaces. *Cognitive Computation*, *12*, 195–205.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. arXiv preprint arXiv:1706.03762

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y. et al. (2017). Graph attention networks. *Stat*, *1050*(20), 10–48550.

Wan, L., Fu, L., Li, C., & Li, K. (2024). Flexible job shop scheduling via deep reinforcement learning with meta-path-based heterogeneous graph neural network. *Knowledge-Based Systems*, *296*, 111940.

Wang, R., Wang, G., Sun, J., Deng, F., & Chen, J.(2023). Flexible job shop scheduling via dual attention network-based reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*,*35*(3), 3091–3102.

Wei, L., He, J., Guo, Z., & Hu, Z. (2023). A multi-objective migrating birds optimization algorithm based on game theory for dynamic flexible job shop scheduling problem. *Expert Systems with Applications*, *227*, 120268.

Xie, J., Li, X., Gao, L., & Gui, L. (2023). A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems. *Journal of Manufacturing Systems*, *71*, 82–94.

Xu, Y., Zhang, M., Yang, M., & Wang, D. (2024). Hybrid quantum particle swarm optimization and variable neighborhood search for flexible job-shop scheduling problem. *Journal of Manufacturing Systems*, *73*, 334–348.

Yuan, E., Wang, L., Cheng, S., Song, S., Fan, W., & Li, Y. (2024). Solving flexible job shop scheduling problems via deep reinforcement learning. *Expert Systems with Applications*, *245*, 123019.

Yue, L., Peng, K., Ding, L., Mumtaz, J., Lin, L., & Zou, T. (2024). Two-stage double deep q-network algorithm considering external non-dominant set for multi-objective dynamic flexible job shop scheduling problems. *Swarm and Evolutionary Computation*, *90*, 101660.

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020a). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, *33*, 1621–1632.

Zhang, G., Hu, Y., Sun, J., & Zhang, W. (2020b). An improved genetic algorithm for the flexible job shop scheduling problem with multiple time constraints. *Swarm and Evolutionary Computation*, *54*, 100664.

Zhang, W., Zhao, F., Li, Y., Du, C., Feng, X., & Mei, X. (2024a). A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem. *Journal of Manufacturing Systems*, *74*, 329–345.

Zhang, Y., Zhu, H., & Tang, D. (2020c). An improved hybrid particle swarm optimization for multi-objective flexible job-shop scheduling problem. *Kybernetes*, *49*(12), 2873–2892.

Zhang, Z., Fu, Y., Gao, K., Pan, Q., & Huang, M. (2024b). A learning-driven multi-objective cooperative artificial bee colony algorithm for distributed flexible job shop scheduling problems with preventive maintenance and transportation operations. *Computers & Industrial Engineering*, *196*, 110484.

Zhu, N., Gong, G., Lu, D., Huang, D., Peng, N., & Qi, H. (2024). An effective reformative memetic algorithm for distributed flexible job-shop scheduling problem with order cancellation. *Expert Systems with Applications*, *237*, 121205.